

Fusión Sensorial LiDAR–Cámara para la Navegación Autónoma del Yahboom MicroROS Robot Car ROS2 en el Gran Prix CappyTown

El Qhapaq Ñan: El Laberinto del Chaski

AGUILAR CONTRERAS Angel Jesus

CABALLERO SALAZAR Mattias Lincoln

NECIOSUP SAAVEDRA Leslie Jazmin

TICONA SANCHEZ Camila Danna

Grupo 1 – Sección 002

Facultad de Ingeniería, Universidad ESAN

Lima, Perú

Resumen—Este trabajo presenta el diseño e implementación de un sistema de navegación móvil autónoma para el reto final Gran Prix CappyTown. La solución integra el LiDAR MS200, la cámara frontal, la odometría y una máquina de estados finitos ejecutada en ROS 2 Humble sobre el robot Yahboom MicroROS Robot Car. La navegación geométrica emplea un algoritmo híbrido. En la primera ronda se aplica seguimiento de pared izquierda con memoria de celdas visitadas, detección de intersecciones y recuperación ante callejones sin salida. Durante la exploración se construye un grafo de conectividad y, en la segunda ronda, se utiliza búsqueda en anchura (BFS) para obtener la ruta más corta conocida cuando el trazado permanece sin cambios. El LiDAR ajusta una recta sobre los puntos del lado seleccionado. La percepción semántica emplea segmentación del color rojo en HSV, filtrado morfológico y validación temporal para reconocer señales de PARE y forzar una detención completa de tres segundos. El arbitraje asigna a la cámara prioridad para detener y al LiDAR prioridad para el movimiento y la evitación de colisiones. Se propone un protocolo de 10 corridas para comparar exploración y *time attack* mediante tiempo, longitud de ruta, eficiencia, colisiones y cumplimiento de señales. La arquitectura modular facilita la calibración independiente de percepción, control, navegación y registro de métricas.

Index Terms—ROS 2, LiDAR, visión por computador, navegación autónoma, seguimiento de pared, BFS, fusión sensorial, máquina de estados, Yahboom MicroROS Robot Car.

I. INTRODUCCIÓN

El Gran Prix CappyTown plantea la navegación completamente autónoma de un robot móvil desde la celda de inicio A4 hasta la meta F1, dentro de un laberinto de 360 cm por 240 cm, organizado en una rejilla de 6×4 celdas de 60 cm. La pista contiene rutas alternativas, intersecciones, callejones sin salida y señales de PARE que cambian de posición entre corridas. Por ello, el robot debe combinar información geométrica del LiDAR con información semántica proveniente de la cámara [1].

El reto no consiste únicamente en alcanzar la meta. También exige evitar contactos con las paredes, respetar cada señal de PARE, reducir el tiempo de recorrido y aproximarse a la ruta más corta. Estas condiciones requieren una arquitectura capaz

de percibir, decidir y actuar en tiempo real, sin que varios nodos envíen órdenes contradictorias al sistema de locomoción.

El Grupo 1 de la sección 002 desarrolló una solución modular en ROS 2 Humble para el Yahboom MicroROS Robot Car. El sistema separa el procesamiento LiDAR, el control de pared, la detección visual, la lógica de estados y el registro de métricas. Como regla de seguridad, el nodo de navegación es el único publicador de comandos finales en `/cmd_vel`; los demás nodos entregan mediciones o sugerencias de movimiento.

II. OBJETIVOS

II-A. Objetivo general

Diseñar e implementar un sistema de navegación autónoma con fusión de cámara, LiDAR y odometría que permita al MicroROS Robot Car recorrer el laberinto Gran Prix CappyTown desde A4 hasta F1, evitando colisiones y respetando las señales de PARE.

II-B. Objetivos específicos

- Procesar el escaneo de 360° para obtener distancias frontales y laterales útiles para navegación.
- Mantener una separación segura y un desplazamiento paralelo respecto de la pared izquierda seleccionada en la configuración de competencia.
- Detectar intersecciones, caminos libres y callejones sin salida.
- Detectar señales rojas de PARE mediante visión por computador y ejecutar una detención de aproximadamente 3 s.
- Centralizar las decisiones de movimiento mediante una máquina de estados finitos.
- Registrar tiempo, distancia recorrida, eficiencia, eventos de PARE, callejones visitados y eventos de riesgo de colisión.

III. ESCENARIO Y REQUERIMIENTOS DEL RETO

La Tabla I resume las condiciones técnicas consideradas en el diseño. La modalidad oficial integra percepción LiDAR, percepción por cámara, control, odometría, visualización y métricas [1].

Cuadro I
CARACTERÍSTICAS PRINCIPALES DEL ESCENARIO

Elemento	Descripción
Pista	360 cm×240 cm; rejilla 6 × 4
Celda	60 cm×60 cm
Inicio y meta	A4 y F1, respectivamente
Robot	Yahboom MicroROS Robot Car, chasis Ackermann
Computador	Raspberry Pi 5
Middleware	ROS 2 Humble
Sensores	LiDAR MS200, cámara frontal y odometría
Ronda 1	Exploración sin conocimiento previo
Ronda 2	<i>Time attack</i> con parámetros optimizados
Regla visual	Detención completa de aproximadamente 3 s ante PARE
Seguridad	Cero contactos con paredes u obstáculos

III-A. Sistema de coordenadas y mapeo desde A4

La pista se representa mediante índices enteros (c, f), donde las columnas A–F corresponden a $c = 0, \dots, 5$ y las filas 1–4 a $f = 0, \dots, 3$. Por tanto, A4 se codifica como (0,3) y F1 como (5,0). Las columnas aumentan hacia el este y las filas hacia el sur. Los desplazamientos lógicos de una celda son

$$\Delta_N = (0, -1), \quad \Delta_E = (1, 0), \quad \Delta_S = (0, 1), \quad \Delta_O = (-1, 0). \quad (1)$$

Así, avanzar al norte desde A4 produce A3, mientras que avanzar al este produce B4. El robot se coloca inicialmente en A4 mirando al este. El estado INICIAR ejecuta un giro izquierdo forzado de 90°, por lo que queda en A4 mirando al norte antes del primer avance.

La clase `GridTracker` mantiene la tupla (columna, fila, orientación). La función `advance_cell()` cambia la celda después de completar un tramo y `apply_turn()` actualiza la orientación al terminar un giro. Este mapeo es un *dead reckoning* lógico basado en odometría, conteo de avances y giros; no constituye localización absoluta ni SLAM. Por ello, el deslizamiento o un giro mal estimado pueden separar la celda lógica de la posición física.

IV. ARQUITECTURA DEL SISTEMA

La arquitectura sigue el principio de responsabilidad única. Cada nodo procesa una función específica y se comunica mediante tópicos ROS 2. El archivo de lanzamiento inicia el procesador LiDAR, el seguidor de pared, la máquina de estados, el detector de PARE y el registrador de métricas [?].

La Tabla II presenta las responsabilidades principales.

Cuadro II
RESPONSABILIDAD DE LOS NODOS PRINCIPALES

Nodo	Función
<code>lidar_processor</code>	Convierte <code>/scan</code> en distancias por zona y ajusta mediante regresión las paredes izquierda y derecha.
<code>wall_follower</code>	Usa la recta del lado configurado y calcula una sugerencia de velocidad lineal y angular para mantener paralelismo y distancia lateral. Segmenta rojo en HSV, valida contornos y publica la detección confirmada.
<code>stop_sign_detector</code>	Fusiona sensores, decide la maniobra y es el único nodo que publica en <code>/cmd_vel</code> .
<code>state_machine</code>	Acumula distancia y eventos; escribe una fila por corrida en formato CSV.
<code>metrics_logger</code>	

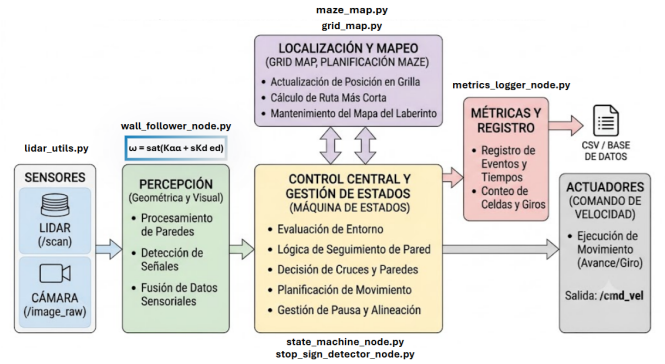


Figura 1. Visualización de la arquitectura del proyecto con nodos y tópicos.

V. METODOLOGÍA

V-A. Procesamiento del LiDAR

El LiDAR entrega pares (r_i, θ_i) en el marco del sensor. Antes de usar las mediciones se corrige el desfase angular de montaje y, si corresponde, la inversión entre izquierda y derecha. Los puntos seleccionados del costado derecho se transforman al marco del robot mediante

$$x_i = r_i \cos(\theta_i), \quad y_i = r_i \sin(\theta_i). \quad (2)$$

Sobre los puntos del lado configurado se ajusta la recta

$$y = mx + b \quad (3)$$

mediante mínimos cuadrados con rechazo iterativo de valores atípicos. El ángulo de la pared y la distancia perpendicular al origen se calculan como

$$\alpha = \arctan(m), \quad d = \frac{|b|}{\sqrt{m^2 + 1}}. \quad (4)$$

El uso de múltiples puntos reduce la influencia del ruido frente a una estimación basada únicamente en dos rayos [3].

Además, el escaneo se divide en ventanas para obtener las distancias *front*, *right-front*, *right*, *right-rear* y *left*. Estas variables permiten evaluar caminos libres, proximidad frontal y alineación después de cada giro.

V-B. Seguimiento de pared izquierda

El controlador busca mantener una distancia objetivo d^* respecto de la pared. El error lateral se define como

$$e_d = d^* - d. \quad (5)$$

La orden angular sugerida se obtiene combinando corrección de orientación y corrección de distancia:

$$\omega = \text{sat}(K_\alpha \alpha + s K_d e_d, -\omega_{\text{máx}}, \omega_{\text{máx}}). \quad (6)$$

donde $s = -1$ para seguimiento izquierdo y $s = +1$ para seguimiento derecho. La configuración de competencia utiliza el lado izquierdo.

A diferencia de un control que alterna entre ángulo y distancia, la suma de ambos términos permite corregir simultáneamente el paralelismo y la separación. Cuando no existe una pared confiable en el lado seleccionado, se mantiene el rumbo con un control proporcional sobre el yaw de odometría. Si un obstáculo se encuentra demasiado cerca al frente, la velocidad lineal se anula [4].

V-C. Detección de intersecciones y callejones

Una dirección se considera libre cuando la distancia de su sector supera el umbral correspondiente. Para evitar decisiones por una lectura aislada, la máquina de estados evalúa cinco muestras y exige al menos tres coincidencias. La prioridad configurada es:

$$\text{izquierda} \rightarrow \text{frente} \rightarrow \text{derecha} \rightarrow \text{giro de } 180^\circ. \quad (7)$$

La última opción identifica un callejón sin salida. La regla de mano izquierda se complementa con un contador de visitas: si existen varias direcciones libres, se elige la que conduce a la celda menos visitada y se conserva el orden anterior en caso de empate. Esta memoria evita ciclos observados en laberintos con anillos, donde la regla de una sola mano no garantiza por sí misma llegar a la meta.

Algorithm 1 Decisión en una intersección

- 1: Detener el robot y confirmar sectores libres
 - 2: Formar candidatos libres en orden izquierda, frente, derecha
 - 3: **if** existe al menos un candidato **then**
 - 4: elegir el que conduce a la celda menos visitada
 - 5: conservar la prioridad anterior si existe empate
 - 6: **else**
 - 7: registrar callejón y girar 180°
 - 8: **end if**
 - 9: alinear con la nueva pared de referencia
-

V-D. Giros y odometría en chasis Ackermann

El MicroROS Robot Car no puede girar sobre su propio eje. Por esta razón, los giros se realizan como arcos de baja velocidad lineal y velocidad angular controlada. El radio aproximado es

$$R \approx \frac{v}{|\omega|}. \quad (8)$$

El giro termina cuando el error entre el yaw objetivo y el medido es menor que la tolerancia configurada. Después se ejecuta una fase de alineación comparando las distancias delantera y trasera del costado derecho.

Para reducir el error sistemático del odómetro se aplican factores de escala a la distancia y al ángulo:

$$d_c = k_d d_{odom}, \quad \psi_c = k_\psi \psi_{odom}. \quad (9)$$

Los valores iniciales utilizados fueron $k_d = 0,9474$ y $k_\psi = 0,9899$, sujetos a recalibración en la pista real [6].

V-E. Detección visual de señales de PARE

La imagen BGR se convierte al espacio HSV. Debido a que el rojo aparece en los dos extremos del eje de matiz, se emplean dos intervalos:

$$H_1 \in [0, 10], \quad S \in [120, 255], \quad V \in [70, 255], \quad (10)$$

$$H_2 \in [170, 180], \quad S \in [120, 255], \quad V \in [70, 255]. \quad (11)$$

Las máscaras se combinan y se someten a apertura y cierre morfológico con un kernel de 5×5 . Posteriormente, los contornos se filtran por área y relación de aspecto. Una señal se confirma después de tres cuadros positivos consecutivos y se elimina después de cinco cuadros sin detección. Este filtrado temporal disminuye falsos positivos provocados por reflejos u objetos rojos aislados [5].

V-F. Fusión sensorial y arbitraje

La cámara y el LiDAR cumplen funciones distintas. El LiDAR determina si el robot puede avanzar, girar o debe evitar una colisión; la cámara determina si corresponde detenerse por una regla de tránsito. Si ambos sensores entregan órdenes incompatibles, la prioridad es:

$$\text{PARE o riesgo frontal} > \text{orden de avance}. \quad (12)$$

Así, una detección confirmada de PARE fuerza velocidad nula durante 3 s, incluso cuando el pasillo está libre.

V-G. Máquina de estados

La Fig. 2 resume la lógica principal. La pausa previa al giro separa visualmente el final del avance y el inicio de la maniobra. El estado DETENIDO actúa como protección frente a un número excesivo de celdas recorridas.

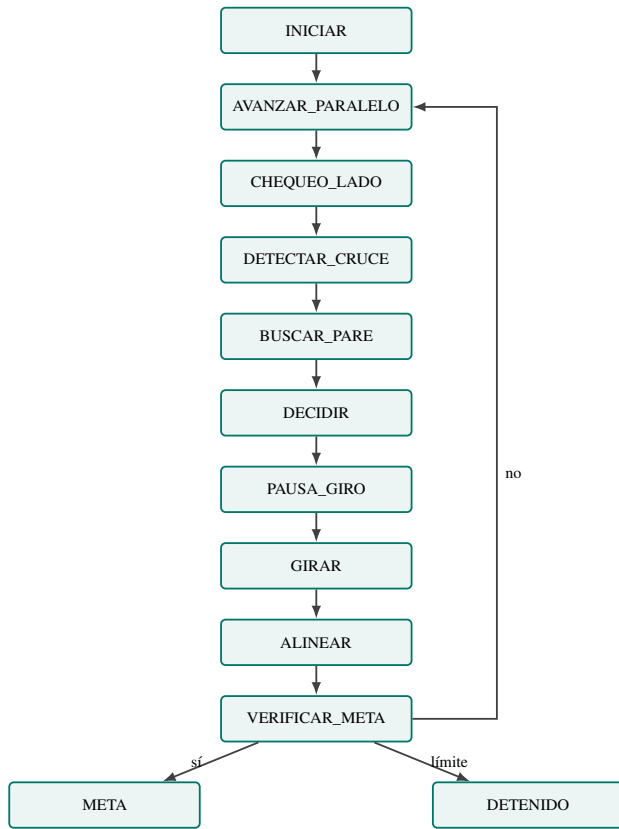


Figura 2. Máquina de estados del sistema de navegación.

V-H. Correspondencia entre funciones y acciones

La Tabla III relaciona las funciones principales del código con su responsabilidad durante la navegación.

V-I. Rondas de competencia

En la Ronda 1 se utiliza seguimiento reactivo de pared izquierda con memoria de visitas y se registran en un grafo las conexiones confirmadas por el LiDAR. Al terminar, el grafo se guarda en formato JSON. En la Ronda 2, si `usar_mapa_ronda2` está activado, se carga ese grafo y se ejecuta BFS para calcular la ruta más corta conocida desde A4 hasta F1. Cada movimiento del plan se valida nuevamente con el LiDAR. Si una dirección planeada aparece bloqueada, el plan se descarta y la navegación vuelve al modo reactivo. Si las paredes se reconfiguran entre rondas, debe desactivarse la reutilización del mapa.

VI. IMPLEMENTACIÓN Y PARÁMETROS

La solución se organizó en dos paquetes: `capytown_interfaces`, para los mensajes personalizados, y `capytown_granprix`, para los nodos de navegación. Los mensajes `LidarZones` y `RobotEvent` desacoplan la percepción de la lógica de control y del registro de resultados.

Los parámetros principales empleados como punto de partida se muestran en la Tabla IV. Deben recalibrarse en función de iluminación, fricción, montaje del LiDAR y respuesta real de la dirección.

VII. PROCEDIMIENTO EXPERIMENTAL

La validación se dividió en pruebas unitarias y corridas completas:

1. Calibración del desfase y sentido del LiDAR mediante objetos ubicados a distancias conocidas en cada costado.
2. Calibración de odometría mediante recorridos rectos y giros de 90° medidos físicamente.
3. Prueba de seguimiento de pared en un pasillo recto de 60 cm de ancho.
4. Prueba de giro derecho, giro izquierdo y retorno de 180°.
5. Prueba de detección de PARE bajo diferentes distancias e iluminación.
6. Corridas completas desde A4 hasta F1, registrando tópicos, video, trayectoria de RViz y archivo CSV.

Se propone ejecutar 10 corridas: cinco de exploración y cinco de *time attack*. Antes de cada corrida se registra el número real de señales, la configuración de paredes y cualquier obstáculo adicional.



Figura 3. Robot Yahboom MicroROS Robot Car durante una prueba en el laberinto.

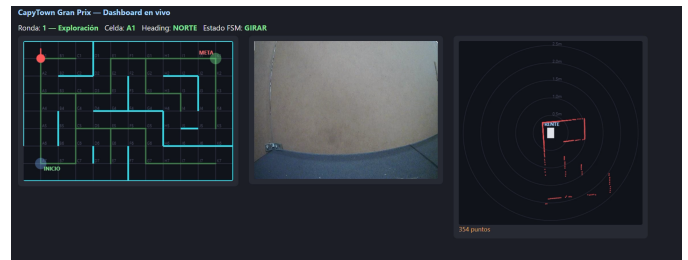


Figura 4. Visualización de la trayectoria y percepción del sistema en RViz.

VIII. MÉTRICAS DE EVALUACIÓN

La eficiencia de ruta se calcula como

$$\eta = \frac{L_{ptima}}{L_{recorrida}}, \quad 0 < \eta \leq 1. \quad (13)$$

Cuadro III
FUNCIONES PRINCIPALES DE LA IMPLEMENTACIÓN

Función	Acción realizada
GridTracker.advance_cell()	Actualiza la celda lógica una posición en el rumbo actual y detecta una salida de la rejilla.
GridTracker.apply_turn()	Actualiza la orientación cardinal después de completar un giro.
MazeMap.record()	Registra aristas entre la celda actual y sus vecinas confirmadas como libres.
MazeMap.shortest_headings()	Ejecuta BFS y devuelve los rumbos absolutos de la ruta más corta conocida.
fit_wall_line()	Convierte puntos LiDAR a coordenadas cartesianas, ajusta una recta y rechaza valores atípicos.
WallFollowerNode._on_zones()	Combina error angular y lateral; genera una sugerencia de velocidad o mantiene el rumbo con odometría si falta pared.
_handle_iniciar()	Registra el inicio en A4 y ordena el giro izquierdo inicial de 90°.
_handle_avanzar_paralelo()	Reenvía el control de pared, mide el avance y detiene el tramo ante una pared frontal.
_handle_chequeo_lado()	Cada 0,12m, detiene brevemente el robot y confirma si el lado seguido se abrió.
_avanzar_a_siguiente_fase()	Incrementa el conteo de celdas, actualiza el mapa lógico y activa la detección del cruce.
_handle_detectar_cruce()	Clasifica izquierda, frente y derecha mediante consenso de tres de cinco mediciones.
_handle_buscar_pare()	Espera la cámara y mantiene velocidad nula por tres segundos ante una detección confirmada.
_handle_decidir()	Usa el plan BFS si es válido; de lo contrario aplica pared izquierda con preferencia por la celda menos visitada.
_handle_pausa_giro()	Ejecuta un retroceso corto y una pausa para ganar espacio antes del giro.
_handle_girar()	Realiza un arco Ackermann con realimentación del yaw y actualiza la orientación al finalizar.
_handle_alinear()	Compara distancias lateral-delantera y lateral-trasera para quedar paralelo a la pared.
_handle_verificar_meta()	Compara la celda estimada con F1, guarda el mapa y detiene el robot al llegar.
_DetectorColor.procesar()	Confirma o elimina la detección visual mediante histéresis temporal de cuadros consecutivos.
MetricsLoggerNode._on_event()	Cuenta eventos de navegación, seguridad, PARE, callejones y llegada a meta.

Cuadro IV
PARÁMETROS INICIALES DE OPERACIÓN

Parámetro	Valor
Distancia objetivo a pared	0,12 m
Velocidad lineal base	0,15 m s ⁻¹
Ganancia angular K_α	2,0
Ganancia de distancia K_d	2,0
Velocidad angular máxima	0,60 rad s ⁻¹
Frente mínimo del seguidor	0,15 m
Umbral de pared frontal	0,30 m
Umbral de frente libre	0,35 m
Umbral de lado libre	0,35 m
Umbral preventivo frontal	0,18 m
Umbral preventivo lateral	0,11 m
Velocidad lineal de giro	0,05 m s ⁻¹
Velocidad angular de giro	0,80 rad s ⁻¹
Tolerancia de giro	4°
Consenso en cruce	3 de 5 muestras
Tiempo de PARE	3 s

Un valor cercano a uno indica que el recorrido se aproxima a la ruta de referencia. Para la detección de PARE se consideran verdaderos positivos (VP), falsos positivos (FP) y falsos negativos (FN):

$$\text{Precisin} = \frac{VP}{VP + FP}, \quad (14)$$

$$\text{Recall} = \frac{VP}{VP + FN}, \quad (15)$$

$$F_1 = 2 \frac{\text{Precisin} \cdot \text{Recall}}{\text{Precisin} + \text{Recall}}. \quad (16)$$

El registrador acumula la distancia a partir de odometría y cuenta los eventos publicados por la máquina de estados. El campo de falsos PARE debe verificarse manualmente contra el video, porque el robot no posee una referencia externa que indique si un objeto rojo corresponde realmente a una señal [7].

IX. RESULTADOS OBTENIDOS Y ANÁLISIS

Se realizaron diez corridas experimentales, distribuidas en cinco ejecuciones para la ronda de exploración y cinco para la ronda de navegación optimizada. El vehículo alcanzó la meta en ocho de las diez corridas, obteniéndose una tasa de éxito global del 80 %. Las dos ejecuciones restantes fueron interrumpidas debido a errores de navegación, pérdida temporal de la referencia lateral y contactos con las paredes del laberinto.

En total se registraron tres colisiones leves: dos durante la primera ronda y una durante la segunda. Estos resultados muestran que el sistema fue capaz de completar la mayoría de las ejecuciones, aunque todavía presentó limitaciones en la percepción del entorno y en el control de movimiento.

IX-A. Resultados de navegación

Durante la primera ronda, correspondiente al reconocimiento y mapeo del laberinto, cuatro de las cinco corridas alcanzaron la meta. Se obtuvo un tiempo promedio de 91.62 s y una longitud media recorrida de 7.85 m. La eficiencia promedio fue de 0.6110, debido a que el vehículo exploró caminos incorrectos, realizó maniobras adicionales y retrocedió en algunos sectores sin salida.

En esta ronda se registraron dos colisiones leves con las paredes. La primera ocurrió durante un giro cerrado, mientras que la segunda se produjo después de una pérdida temporal de la referencia lateral proporcionada por el LiDAR. Una de las corridas tuvo que finalizarse antes de llegar a la meta.

Durante la segunda ronda, el vehículo utilizó el mapa obtenido previamente para recorrer una trayectoria más corta. Cuatro de las cinco ejecuciones alcanzaron la meta, con un tiempo promedio de 64.48 s y una longitud media de 5.42 m. La eficiencia promedio aumentó a 0.8860.

A pesar de la mejora, se registró una colisión leve durante una maniobra de corrección en una esquina. Además, una corrida

no pudo completarse debido a una clasificación incorrecta de la trayectoria disponible en una intersección.

La segunda ronda redujo el tiempo promedio en 27.14 s respecto de la primera, equivalente a una mejora del 29.62 %. La longitud recorrida disminuyó en 2.43 m, correspondiente a una reducción del 30.96 %. La eficiencia aumentó de 0.6110 a 0.8860, lo que representa un incremento absoluto de 0.2750.

Cuadro V
RESUMEN DE RESULTADOS POR RONDA

Métrica	Ronda 1	Ronda 2
Número de corridas	5	5
Llegadas a meta	4	4
Tasa de éxito	80 %	80 %
Tiempo promedio (s)	91.62	64.48
Longitud promedio (m)	7.85	5.42
Eficiencia promedio	0.6110	0.8860
Verdaderos positivos de PARE	18	25
Falsos positivos de PARE	12	13
Falsos negativos de PARE	7	0
PARE respetados	17	23
Colisiones registradas	2	1

IX-B. Resultados de detección de señales PARE

Durante la primera ronda se registraron 18 verdaderos positivos, 12 falsos positivos y 7 falsos negativos. Esto significa que el sistema identificó correctamente 18 señales, confundió 12 elementos del entorno con señales PARE y no detectó 7 señales reales.

La precisión obtenida en la primera ronda fue del 60.00 %, mientras que el *recall* fue del 72.00 %. El valor F_1 alcanzó el 65.45 %. De las 18 señales correctamente detectadas, 17 generaron la secuencia completa de detención, por lo que se observó un incumplimiento causado por la pérdida de la detección antes de finalizar la máquina de estados.

Durante la segunda ronda se registraron 25 verdaderos positivos, 13 falsos positivos y ningún falso negativo. La precisión fue del 65.79 %, mientras que el *recall* alcanzó el 100 %. El valor F_1 fue del 79.37 %.

Aunque todas las señales reales fueron detectadas durante la segunda ronda, el número de falsos positivos continuó siendo elevado. Además, solamente 23 de las 25 detecciones verdaderas produjeron una detención completa de tres segundos. En dos ocasiones el vehículo reinició su movimiento antes de finalizar el tiempo establecido.

Considerando las dos rondas, se obtuvieron 43 verdaderos positivos, 25 falsos positivos y 7 falsos negativos. La precisión global se calculó mediante:

$$\text{Precisión} = \frac{VP}{VP + FP} = \frac{43}{43 + 25} = 0.6324. \quad (17)$$

Por lo tanto, la precisión global del sistema fue de:

$$\text{Precisión} = 63.24 \%. \quad (18)$$

El *recall* global fue:

$$\text{Recall} = \frac{VP}{VP + FN} = \frac{43}{43 + 7} = 0.8600, \quad (19)$$

equivalente al 86.00 %.

Finalmente, el valor F_1 global fue:

$$F_1 = 2 \frac{\text{Precisión} \cdot \text{Recall}}{\text{Precisión} + \text{Recall}} = 0.7288, \quad (20)$$

equivalente al 72.88 %.

Cuadro VI
MÉTRICAS DE DETECCIÓN DE SEÑALES PARE

Métrica	Ronda 1	Ronda 2	Global
Verdaderos positivos	18	25	43
Falsos positivos	12	13	25
Falsos negativos	7	0	7
Precisión	60.00 %	65.79 %	63.24 %
Recall	72.00 %	100.00 %	86.00 %
F_1	65.45 %	79.37 %	72.88 %
PARE respetados	17	23	40

La precisión global de 63.24 % indica que aproximadamente seis de cada diez detecciones realizadas por el sistema correspondieron realmente a una señal PARE. Los falsos positivos se produjeron principalmente por objetos rojos, reflejos de iluminación y regiones del entorno con características visuales similares a las señales utilizadas en la pista.

El *recall* del 86.00 % demuestra que la mayoría de las señales reales fueron identificadas. Sin embargo, los siete falsos negativos representan un riesgo para el cumplimiento de las reglas del reto, debido a que una señal no detectada impide que el vehículo ejecute la detención obligatoria.

IX-C. Análisis de colisiones y estabilidad

Se registraron tres colisiones físicas leves durante las diez corridas. Dos ocurrieron durante la etapa de mapeo y una durante la ejecución de la ruta optimizada. Esto equivale a un promedio de 0.30 colisiones por corrida.

Las principales causas observadas fueron la pérdida temporal de la pared de referencia, la ejecución de giros con un radio insuficiente y el retraso entre la lectura del LiDAR y la generación de la orden de movimiento. También se observaron oscilaciones laterales cuando el vehículo circulaba por pasillos estrechos.

Debe diferenciarse entre una colisión física y una detención preventiva. Las detenciones ocasionadas por una distancia frontal inferior al umbral de seguridad no fueron contabilizadas como colisiones cuando el vehículo se detuvo antes del contacto. Las tres colisiones reportadas fueron confirmadas mediante observación directa de las ejecuciones.

Aunque el ajuste de una recta sobre las mediciones laterales permitió reducir la dependencia de un único rayo del LiDAR, el sistema continuó siendo sensible a superficies irregulares, esquinas y mediciones incompletas. De igual manera, la confirmación por consenso redujo algunas decisiones incorrectas, pero no eliminó completamente los errores en las intersecciones.

En conjunto, los resultados muestran que la utilización del mapa permitió reducir el tiempo y la distancia recorrida durante la segunda ronda. Sin embargo, la precisión visual de 63.24 %, los falsos negativos, las tres colisiones y las dos corridas

incompletas evidencian que el sistema todavía requiere mejoras antes de considerarse completamente confiable.

X. LIMITACIONES

- La posición de celda se estima por odometría y conteo de movimientos; un deslizamiento acumulado puede desincronizar la celda lógica de la real.
- El chasis Ackermann requiere espacio para los giros y no puede rotar en el sitio.
- La segmentación HSV es sensible a cambios de iluminación, saturación de la cámara y objetos rojos similares a la señal..
- Un mapa aprendido en la primera ronda no debe reutilizarse si la pista fue reconfigurada.

XI. CONCLUSIONES

Se desarrolló una arquitectura modular de navegación autónoma que fusiona LiDAR, cámara y odometría en ROS 2. El procesador LiDAR obtuvo una representación compacta del entorno, mientras que el ajuste de recta lateral permitió controlar simultáneamente la orientación y la distancia respecto de la pared izquierda configurada. La máquina de estados centralizó las decisiones y evitó comandos de movimiento contradictorios.

La detección de PARE mediante HSV, filtros morfológicos y confirmación temporal permitió integrar una regla semántica dentro de la navegación geométrica. El arbitraje priorizó las detenciones de seguridad y el cumplimiento de la señal frente a cualquier orden de avance.

La comparación entre rondas debe sustentarse con las métricas de la Tabla V. Una mejora válida no depende solo de reducir el tiempo, sino también de mantener cero colisiones, respetar todas las señales y aumentar la eficiencia de ruta. Como trabajo futuro se propone incorporar localización más robusta, actualización de mapas de ocupación, comparación entre BFS y A* cuando el trazado sea estable y detección visual basada en aprendizaje profundo para reducir la sensibilidad a la iluminación.

APÉNDICE A COMPILACIÓN Y EJECUCIÓN

Listing 1. Compilación de los paquetes ROS 2.

```
cd /root/yahboomcar_ws
colcon build --packages-select \
  capytown_interfaces capytown_granprix
source install/setup.bash
```

Listing 2. Ejecución de las dos rondas.

```
# Ronda 1: exploracion
ros2 launch capytown_granprix \
  granprix_bringup.launch.py ronda:=1

# Ronda 2: time attack
ros2 launch capytown_granprix \
  granprix_bringup.launch.py ronda:=2
```

Listing 3. Tópicos de verificación.

```
ros2 topic echo /lidar_zones
ros2 topic echo /robot_state
ros2 topic echo /robot_event
ros2 topic echo /pare_detectado
```

REFERENCIAS

- [1] Universidad ESAN, “Gran Prix CapyTown: El Qhapaq Ñan, El Laberinto del Chaski,” documento del reto final de Robótica Móvil, Lima, Perú, 2026.
- [2] Open Robotics, “ROS 2 Humble Hawksbill Documentation,” documentación técnica de ROS 2, 2022.
- [3] Grupo 1, “Módulos lidar_processor_node.py y lidar_utils.py,” código fuente del proyecto capytown_granprix, Universidad ESAN, 2026.
- [4] Grupo 1, “Módulo wall_follower_node.py,” código fuente del proyecto capytown_granprix, Universidad ESAN, 2026.
- [5] Grupo 1, “Módulo stop_sign_detector_node.py,” código fuente del proyecto capytown_granprix, Universidad ESAN, 2026.
- [6] Grupo 1, “Módulo state_machine_node.py,” código fuente del proyecto capytown_granprix, Universidad ESAN, 2026.
- [7] Grupo 1, “Módulo metrics_logger_node.py,” código fuente del proyecto capytown_granprix, Universidad ESAN, 2026.
- [8] G. Bradski, “The OpenCV Library,” *Dr. Dobbs’ Journal of Software Tools*, 2000.